

Secure Token-Efficient Autonomous Coding Agent

Complete System Architecture

Version: 1.0 MVP architecture

Date: 2026-09-28

Status: Implementation baseline

Primary goals: safe repository modification, efficient model context, measurable reliability, and controlled autonomy.

1. Executive Summary

This system is an autonomous coding agent that receives a software-engineering task, analyzes the repository, plans a solution, retrieves only relevant context, invokes an OpenAI-compatible Nebius Token Factory model, executes tools inside a restricted sandbox, validates the result, and returns a concise outcome.

The architecture is deliberately split into two control planes:

1. **Reasoning plane** — task analysis, planning, context retrieval, model routing, and result critique.
2. **Execution and security plane** — policy enforcement, command authorization, secret isolation, filesystem/network controls, sandbox execution, audit logging, and approval handling.

Security invariant: The model never receives unrestricted host access. Every file read, file write, shell command, network request, and Git mutation is mediated by a policy-controlled tool boundary.

Efficiency invariant: The full repository is never sent to the model by default. Context is indexed, ranked, compressed, cached, and budgeted before inference.

1.1 MVP success criteria

- Complete representative coding tasks without direct host access.
- Block attempts to read credentials, escape the sandbox, access unrelated repositories, or perform unauthorized network/Git actions.

- Reduce input tokens relative to a full-repository baseline while maintaining task success.
 - Produce reproducible traces for every model decision and tool action.
 - Stop safely on repeated failures, policy violations, timeouts, or uncertainty.
-

2. Scope and Non-Goals

In scope

- One repository per isolated task run.
- Python, JavaScript/TypeScript, and common configuration formats in the MVP.
- Read/write repository tools, shell commands, test execution, and controlled Git inspection.
- Nebius Token Factory through its OpenAI-compatible API.
- Model routing among at least two tested NVIDIA Nemotron variants.
- Prompt-injection detection and untrusted-content labeling.
- Filesystem, shell, network, secret, and Git policy enforcement.
- Human approval for high-impact operations.
- Evaluation harness for quality, efficiency, and security.

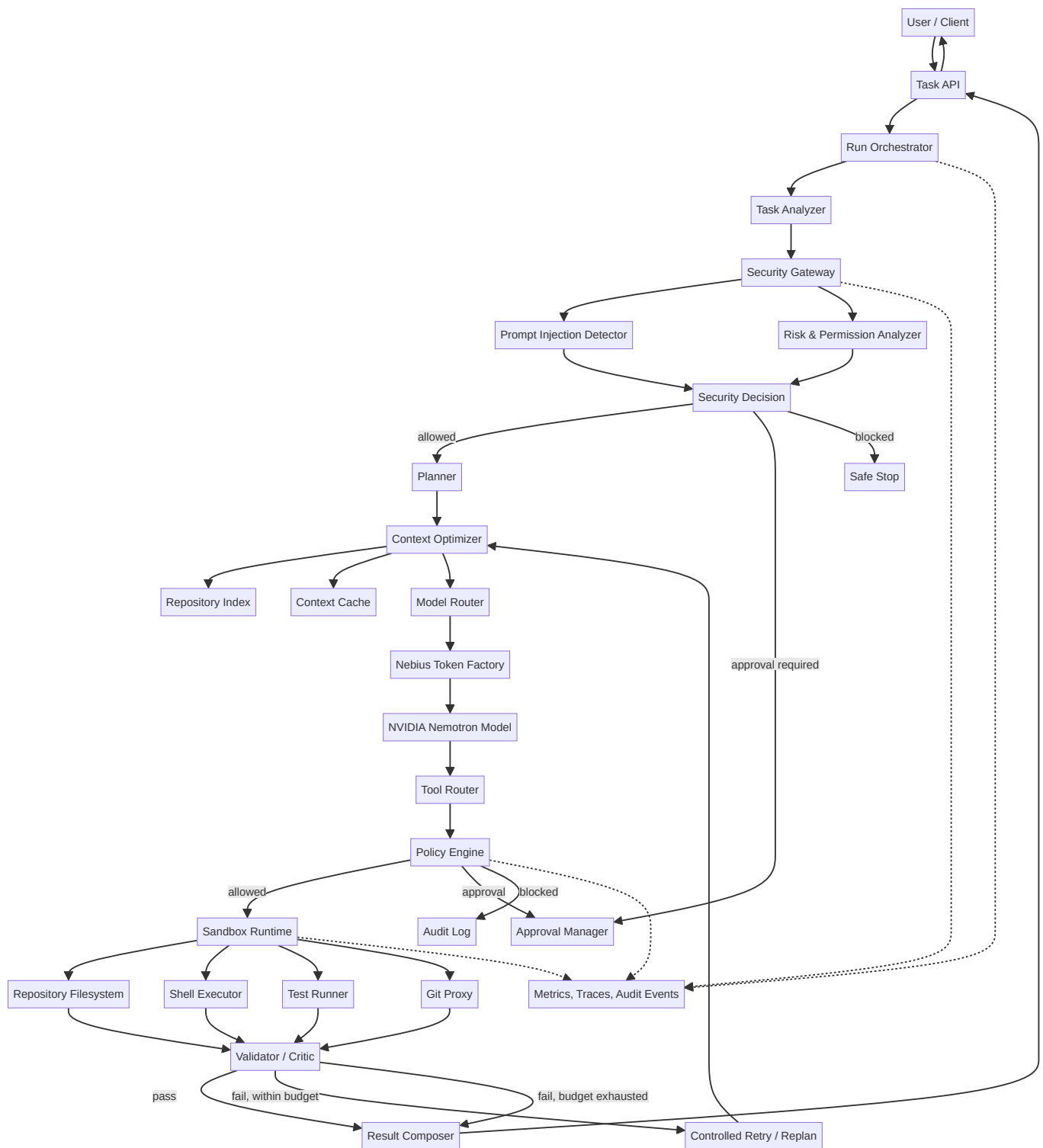
Out of scope for MVP

- Autonomous production deployment.
 - Unreviewed `git push`, account changes, billing changes, or credential rotation.
 - Multi-tenant shared execution without strong tenant isolation.
 - Arbitrary internet browsing by the model.
 - Treating a detector or model classifier as a complete security boundary.
 - Automatic installation from unknown package sources.
-

3. Architectural Principles

Principle	Application
Least privilege	Grant only the repository paths, commands, destinations, and time needed for the task.
Default deny	Unknown commands, paths, destinations, and capabilities are denied.
Policy before execution	Tool requests are validated before reaching the runtime.
Defense in depth	Combine parsing, allowlists, identity, sandboxing, network controls, runtime monitoring, and audit logs.
Untrusted repository content	README files, comments, fixtures, generated files, and tool output may contain prompt injection.
Deterministic control	The model proposes actions; policy and runtime components decide whether they execute.
Reproducibility	Pin task inputs, repository revision, policies, model, parameters, and tool versions.
Bounded autonomy	Limit steps, retries, tool calls, time, output size, and resource use.
Fail closed	Policy errors, missing authorization, detector uncertainty, and runtime anomalies do not silently fall through.
Measurable claims	Security and token improvements are demonstrated against fixed baselines.

4. High-Level Architecture



4.1 Trust boundaries

1. **Client boundary:** User input is untrusted and authenticated by the Task API.

2. **Model boundary:** Model output is untrusted; it is treated as a proposal, not an instruction with authority.
 3. **Repository boundary:** Repository content is untrusted input and may contain injection attempts.
 4. **Policy boundary:** All capabilities cross the Policy Engine before execution.
 5. **Sandbox boundary:** Runtime access is isolated from the host and unrelated user data.
 6. **Secret boundary:** Raw secrets exist only in the secure proxy or approved runtime path, never in prompts, repository files, or ordinary logs.
 7. **External network boundary:** Egress is denied by default and restricted by destination policy.
-

5. Component Architecture

5.1 Task API and Run Orchestrator

Responsibilities

- Authenticate the caller and validate task input.
- Create an immutable `run_id` and task budget.
- Pin repository revision and policy profile.
- Coordinate state transitions and cancellation.
- Persist run metadata and emit lifecycle events.

Inputs

- Repository identifier or uploaded workspace.
- Task description.
- Optional test command and acceptance criteria.
- Requested autonomy level.
- Time, token, step, and cost budgets.

Outputs

- Run status: queued, analyzing, planning, executing, validating, awaiting approval, completed, failed, blocked, cancelled.

- Final summary, patch/diff, validation results, blocked actions, and trace reference.

5.2 Task Analyzer

Normalizes the task into a structured objective:

```
{
  "objective": "Fix invalid-password handling",
  "constraints": ["preserve API contract", "add regression test"],
  "acceptance_tests": ["pytest tests/auth"],
  "risk_class": "medium",
  "likely_artifacts": ["application code", "tests", "configuration"],
  "required_capabilities": ["repository_read", "repository_write", "test_"]
}
```

It must not grant permissions. It only proposes required capabilities for later policy evaluation.

5.3 Security Gateway

Runs before planning and whenever new content or capabilities enter the loop.

- Labels external/user/repository/model/tool content.
- Detects likely prompt injection.
- Assesses requested capabilities and risk.
- Applies task-level policy profile.
- Creates an explicit security decision with reasons.

Prompt-injection detection is advisory to the policy system; it is not the sole security boundary.

5.4 Planner

Creates a bounded plan containing:

- Hypothesis about the defect or change.
- Files/symbols to inspect.
- Expected tool sequence.
- Tests to run.
- Rollback or stop conditions.

- Estimated token and execution budget.

The planner cannot execute tools directly.

5.5 Context Optimizer

The context optimizer is a separate service/module because token efficiency is a primary research contribution.

Pipeline:



Index fields

- Path, language, size, hash, and revision.
- Imports and exports.
- Classes, functions, methods, symbols, and references.
- Test files and test-to-source relationships.
- Configuration and dependency manifests.
- Embeddings or lexical features where enabled.
- Last-modified metadata and Git history signals.

Ranking features

- Task keyword overlap.
- Symbol/reference graph distance.
- Import/dependency relationship.
- Test failure location.
- Prior tool output and model attention.
- Git history relevance.
- Semantic similarity.
- File type and generated-file penalty.

Compression rules

- Preserve exact code around high-confidence relevant symbols.
- Summarize low-confidence supporting files.

- Elide unchanged boilerplate.
- Replace long tool output with structured failure summaries.
- Never compress away security-relevant evidence, command text, or policy decisions.

Loop detection record

```
{  
  "action_hash": "sha256(...) ",  
  "action_type": "shell",  
  "normalized_command": "pytest tests/auth/test_login.py",  
  "target": "tests/auth/test_login.py",  
  "result_class": "same_failure",  
  "timestamp": "...",  
  "attempt": 3  
}
```

If the same unsuccessful action reaches a configurable threshold, the system must stop, change strategy, or request approval rather than repeat indefinitely.

5.6 Model Router and Nebius Adapter

The adapter exposes a provider-neutral interface:

```
complete(messages, tools, model, parameters, timeout) → ModelResponse
```

The router selects a model using task complexity, budget, observed reliability, and policy-approved model availability.

Task complexity	Candidate route	MVP decision
Simple lookup/edit	Nemotron Nano	Prefer low latency and low cost
Moderate coding task	Nemotron Super	Prefer tool use and coding quality
Complex debugging/architecture	Nemotron Ultra	Use only if evaluation supports quality gain

The exact routing policy must be experimentally validated. Do not assume the largest model is best.

Provider controls

- Request timeout and retry with exponential backoff.
- Idempotency key per inference request.
- Maximum input/output tokens.
- No raw secrets in messages.
- Redacted provider logs.
- Provider health and quota metrics.
- Model/version pinning for evaluations.

5.7 Tool Router

The model can request only typed tools, not arbitrary host calls.

Tool	Purpose	Default access
<code>repo.list</code>	List allowed repository paths	Allow
<code>repo.read</code>	Read approved file ranges	Allow
<code>repo.write</code>	Apply bounded file patch	Allow with validation
<code>shell.run</code>	Run structured command	Policy-dependent
<code>test.run</code>	Run approved test command	Allowlist
<code>git.status</code>	Inspect working tree	Allow
<code>git.diff</code>	Inspect changes	Allow
<code>git.branch</code>	Inspect branches	Allow
<code>git.commit</code>	Create commit	Approval/policy controlled
<code>git.push</code>	Push remote changes	Explicit human approval only
<code>network.fetch</code>	Access approved destination through proxy	Deny by default

Tool requests include the run identity, capability, target, arguments, expected impact, and reason. The Tool Router rejects malformed or unknown schemas before policy evaluation.

5.8 Policy Engine

The Policy Engine is deterministic and independent of the model.

```
Decision = evaluate(  
    principal,  
    task_policy,  
    capability,  
    normalized_arguments,  
    target,  
    risk,  
    runtime_state,  
    approval_state  
)
```

Possible decisions:

- `ALLOW` — execute automatically.
- `ALLOW_WITH_MONITORING` — execute and emit enhanced telemetry.
- `REQUIRE_APPROVAL` — pause before execution.
- `DENY` — do not execute; provide safe reason.
- `QUARANTINE` — isolate artifact or terminate run for investigation.

5.9 Sandbox Runtime

Each run receives a disposable runtime with:

- Non-root execution identity.
- Read/write mount only for the target repository and task temp directory.
- No access to host home, SSH keys, cloud credentials, browser profiles, or unrelated projects.
- CPU, memory, disk, process, file-count, and wall-clock limits.
- Read-only base image where possible.
- Ephemeral workspace and cleanup after completion.
- Runtime syscall/container restrictions appropriate to the deployment platform.

5.10 Validator / Critic

Validation is layered:

1. Patch applies cleanly.

2. Formatting and syntax checks.
3. Static checks and linting.
4. Targeted tests.
5. Relevant broader tests within budget.
6. Security regression checks.
7. Diff review against task constraints.

The Critic returns structured findings rather than raw output:

```
{
  "status": "fail",
  "tests_total": 287,
  "tests_failed": 3,
  "failures": [
    {
      "test": "test_login_invalid_password",
      "expected": "401",
      "actual": "500",
      "traceback_excerpt": "... "
    }
  ],
  "security_findings": [],
  "recommended_action": "inspect exception mapping in auth handler"
}
```

6. Security Architecture

6.1 Filesystem policy

Path class	Access	Notes
Target repository	Read/write	Mount only for the run
Run temp directory	Read/write	Ephemeral and quota-limited
Host home directory	Deny	Never mounted
<code>~/.ssh</code> , <code>~/.aws</code> , credential stores	Deny	Add runtime secret scanning
Unrelated repositories	Deny	Enforce mount and path checks
System files and device nodes	Deny	Container/runtime isolation
Build caches	Isolated	Use task-scoped cache where needed

Path validation must canonicalize symlinks, reject traversal, enforce mount boundaries, and validate both requested and resolved paths.

6.2 Shell command policy

Command classification must use structured parsing plus runtime controls, not regex alone.

Risk	Examples	MVP behavior
Low	<code>ls</code> , <code>cat</code> , <code>git diff</code> , <code>pytest</code>	Automatic if target is within policy
Medium	<code>pip install</code> , <code>npm install</code> , <code>docker build</code>	Policy check, pinned/approved sources, monitoring
High	<code>git push</code> , network configuration, unknown package source	Explicit user approval
Critical	<code>rm -rf</code> , raw disk operations, credential access, host escape	Block and audit

Controls:

- Parse shell syntax into an AST where available.
- Deny command substitution or nested interpreters unless explicitly allowed.
- Resolve executable path and verify approved binary identity.
- Enforce environment-variable allowlist.
- Enforce working-directory and output-path restrictions.
- Apply process, network, and resource limits.
- Capture stdout, stderr, exit code, signals, duration, and file/network effects.

6.3 Network policy

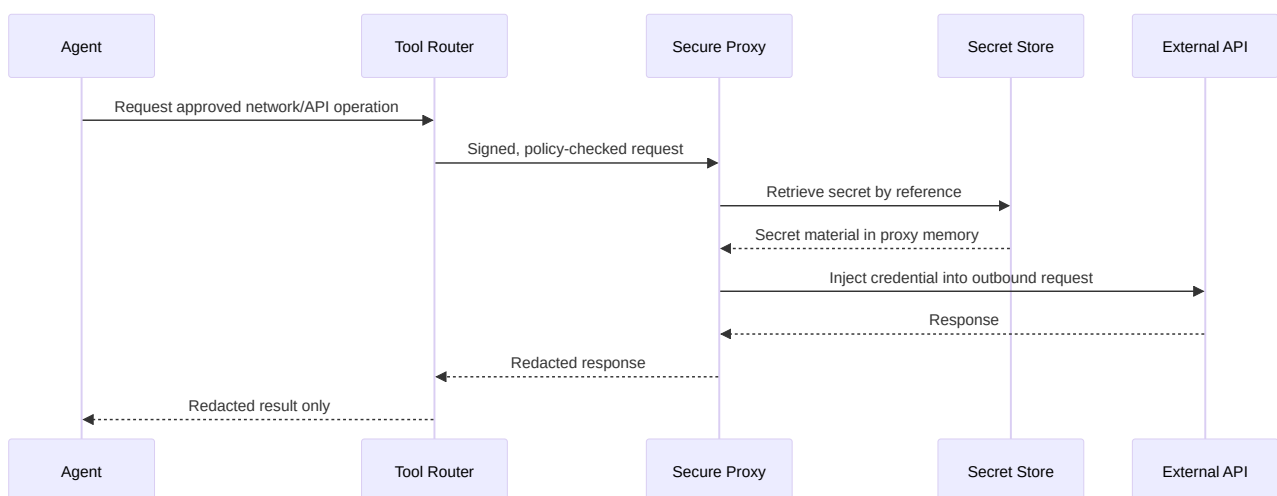
Default: **Internet denied**.

Allow only through an egress proxy/firewall with destination and method controls:

- `github.com` or approved Git host.
- `pypi.org` and approved package mirrors.
- `npmjs.org` and approved package mirrors.
- Nebius API endpoints.
- Approved documentation sources.

DNS, IP, redirect, TLS, HTTP method, response size, and upload controls must be applied. The model must not be able to bypass the proxy using raw sockets or alternate DNS.

6.4 Secret protection



Rules:

- Never place raw API keys in prompts, repository files, command arguments, or ordinary logs.
- Prefer short-lived, scoped credentials.
- Redact secrets from stdout/stderr, traces, diffs, exceptions, and model context.
- Treat environment variables as sensitive unless explicitly allowlisted.
- Scan files and tool output for accidental secret disclosure.

6.5 Git policy

Operation	Default
<code>git status</code> , <code>git diff</code> , <code>git branch</code>	Allowed
<code>git log</code> , local read-only inspection	Allowed within repository
<code>git commit</code>	Approval or configured policy
<code>git reset</code> , destructive history changes	Block or explicit approval
<code>git push</code>	Explicit human approval
Remote URL changes	Block by default

6.6 Prompt-injection handling

Potential injection sources include README files, source comments, test fixtures, generated files, dependency metadata, issue text, shell output, and remote documentation.

Mitigations:

1. Mark every retrieved content block with provenance and trust level.
 2. Separate instructions from repository data in the model message structure.
 3. Tell the model that repository text is data, not authority.
 4. Detect suspicious instruction-like content and escalate risk.
 5. Prevent content from modifying policy or tool schemas.
 6. Require policy validation for every resulting action.
 7. Log detector evidence and final disposition.
-

7. Data Architecture

7.1 Core entities

Entity	Purpose	Key fields
Task	User objective	task_id , description, constraints, requester
Run	One reproducible execution	run_id , task, repo revision, policy, state, budgets
RepositorySnapshot	Immutable input revision	URL/id, commit SHA, file manifest, hash
Plan	Bounded reasoning plan	steps, expected files, acceptance criteria
ContextItem	Retrieved context	path, symbol, provenance, rank, token count
ToolRequest	Model-proposed action	tool, args, risk, timestamp
PolicyDecision	Authorization outcome	decision, reason, policy version
Approval	Human authorization	scope, approver, expiry, decision
Execution	Runtime action	command/tool, exit code, resources, outputs
ValidationResult	Test/critic result	checks, failures, security findings
MetricRecord	Experiment measurement	model, params, tokens, time, cost, success
AuditEvent	Immutable security trace	actor, event, target, decision, evidence

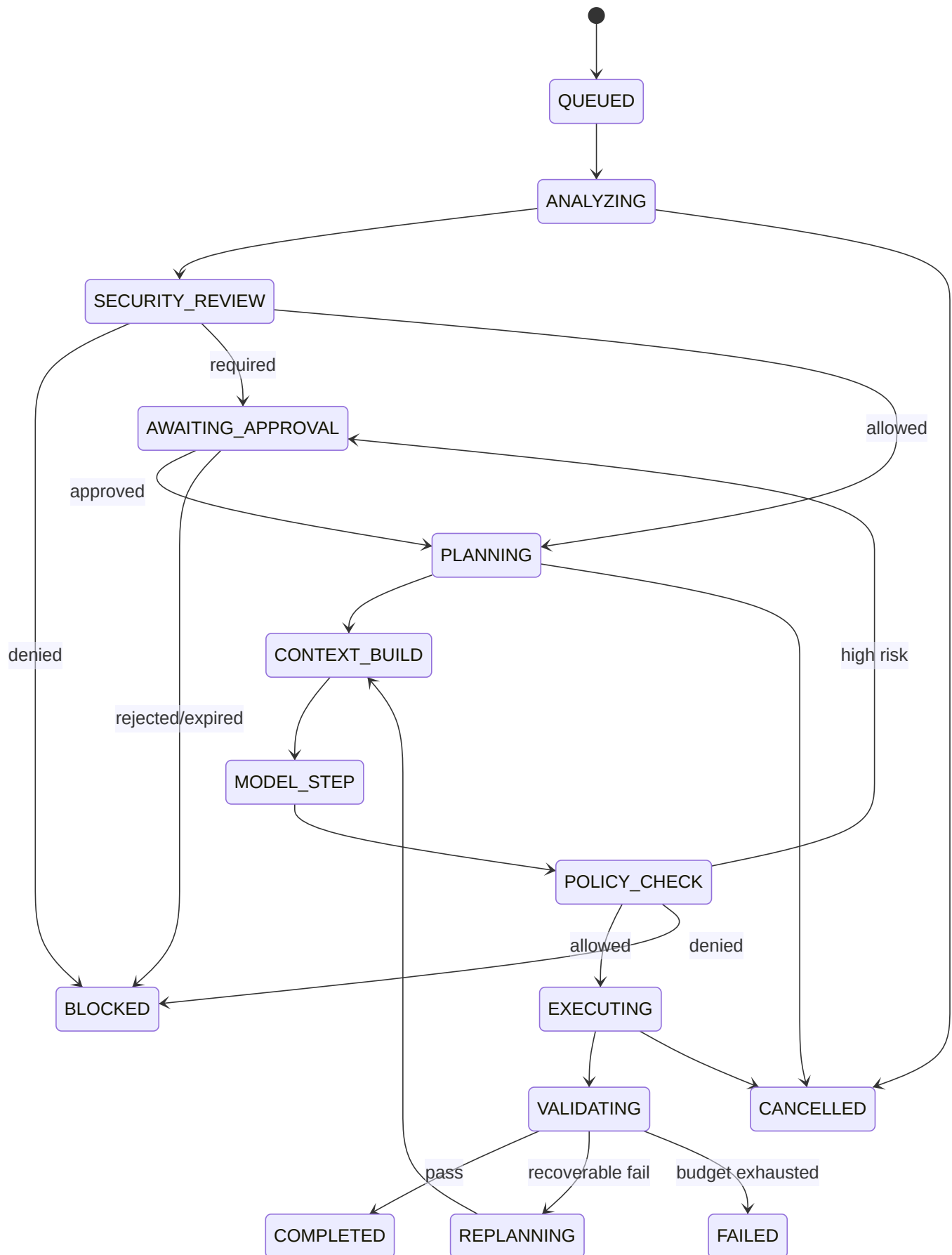
7.2 Storage recommendations

- **Relational database:** task/run state, policies, approvals, metrics, validation records.

- **Object storage:** large logs, patches, repository snapshots, compressed context artifacts.
- **Search/index store:** repository paths, symbols, references, lexical/semantic retrieval features.
- **Cache:** repository summary, file/symbol summaries, test results, provider responses where safe, and normalized tool results.
- **Append-only audit store:** security events and policy decisions with retention controls.

No model context cache may contain unredacted credentials or data outside the run's authorization scope.

7.3 State machine



8. API and Tool Contracts

8.1 Create run

POST /v1/runs

Content-Type: application/json

```
{
  "repository": {
    "source": "workspace://repo",
    "revision": "commit-sha-or-worktree-hash"
  },
  "task": "Fix invalid password handling and add a regression test.",
  "acceptance": ["pytest tests/auth"],
  "autonomy": "sandboxed",
  "budgets": {
    "max_steps": 40,
    "max_tool_calls": 80,
    "max_retries": 3,
    "timeout_seconds": 900,
    "max_input_tokens": 50000,
    "max_output_tokens": 12000
  }
}
```

8.2 Tool request envelope

```
{
  "run_id": "run_123",
  "request_id": "req_456",
  "tool": "repo.write",
  "arguments": {
    "path": "src/auth.py",
    "patch": "*** Begin Patch ..."
  },
  "reason": "map invalid credentials to HTTP 401",
  "expected_risk": "medium",
  "provenance": "model_step_7"
}
```

8.3 Policy decision envelope

```
{
  "decision": "ALLOW_WITH_MONITORING",
  "policy_version": "mvp-1",
  "risk": "medium",
  "constraints": ["repository_mount_only", "no_network"],
  "reason_codes": ["approved_tool", "path_within_workspace"],
  "expires_at": "..."
}
```

9. Observability and Auditability

Every run must emit correlated events using `run_id`, `step_id`, `request_id`, and `trace_id`.

Required metrics

- Task success rate.
- Validation pass rate.
- Input, output, and total tokens.

- Context retrieval size and compression ratio.
- Tool-call count and retry count.
- End-to-end and per-step latency.
- Estimated and actual provider cost.
- Policy denials, approvals, and false-positive blocks.
- Prompt-injection detection rate and disposition.
- Sandbox violations and network denials.
- Resource consumption and timeout rate.

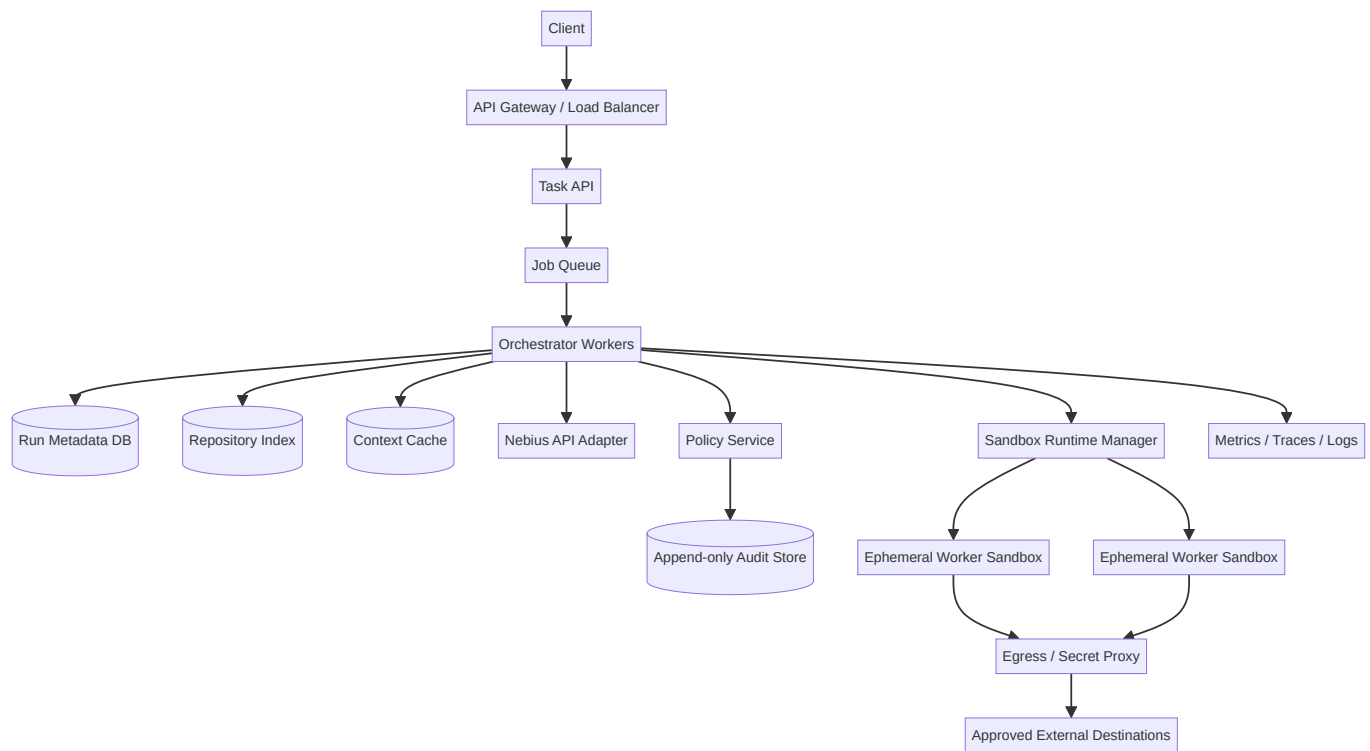
Audit events

Record:

- User/task creation.
- Repository revision and policy version.
- Model/provider/parameters.
- Context items exposed to the model.
- Every proposed tool request.
- Normalized command and target.
- Policy decision and reason.
- Approval request, approver, scope, and expiry.
- Runtime result and redacted output hash.
- Validation result and final patch hash.
- Stop reason.

Audit logs must be append-only, access-controlled, tamper-evident, and redacted.

10. Deployment Architecture



Deployment requirements

- API and orchestration services run outside task sandboxes but cannot directly access repository contents except through approved workspace APIs.
- Sandboxes are created per run or per strongly isolated task lease.
- Sandbox images are versioned, scanned, and pinned.
- Worker-to-sandbox communication uses authenticated, encrypted channels.
- Network egress is enforced below the model/tool layer as a runtime control.
- Database and object-store access uses service identities with separate permissions.
- Production deployment remains a separate, approval-gated system.

Suggested MVP implementation

Area	MVP choice
API/orchestrator	Python service with typed models and an explicit run state machine
Queue	Redis-backed queue or managed equivalent
Metadata	PostgreSQL
Index	SQLite/PostgreSQL for lexical MVP; add vector index only if justified
Cache	Redis or local task-scoped cache
Sandbox	Rootless containers or isolated execution service
Policy	Versioned declarative policy plus deterministic evaluator
Provider	OpenAI-compatible Nebius adapter
Observability	OpenTelemetry-compatible traces, metrics, structured logs
Secrets	Secret manager plus outbound proxy

11. Execution Algorithm

1. Authenticate request and create run record.
2. Pin repository revision and create isolated workspace.
3. Analyze task and derive proposed capabilities.
4. Run initial security review and classify untrusted inputs.
5. Build/update repository index.
6. Create a bounded plan.
7. Retrieve and rank relevant files/symbols.
8. Compress context within the token budget.
9. Select model and parameters using routing policy.
10. Invoke model with typed tools and provenance labels.
11. Validate the model response schema.
12. Send each tool request through Tool Router and Policy Engine.
13. Request approval when required; never infer approval from silence.
14. Execute allowed actions in the sandbox.
15. Capture and compress results; redact secrets.
16. Detect loops, excessive retries, policy drift, and budget exhaustion.
17. Run validator/critic checks.
18. If recoverable, replan with only relevant new evidence.
19. If passing, produce patch, tests, summary, and trace.
20. If failing or blocked, stop safely with an actionable reason.
21. Destroy or quarantine the runtime according to retention policy.

12. Model and Parameter Experiment Plan

Do not claim that a single parameter makes the system secure or reliable. Evaluate the interaction of:

- Model selection.
- Temperature.
- Maximum output tokens.
- Context size and retrieval count.
- Reasoning settings where supported.
- Tool-call limit.
- Retry limit.

- Maximum agent steps.
- Timeout.
- Context compression strategy.

For each run, record:

```
Task, model, model version, parameters, input tokens, output tokens,  
total tokens, tool calls, retries, elapsed time, estimated cost,  
success, security violations, policy denials, approvals, stop reason
```

Use fixed seeds or deterministic settings where supported, repeated trials, and confidence intervals for comparisons.

13. Evaluation Strategy

13.1 Functional task suite

1. Fix a Python bug.
2. Add a function.
3. Add a unit test.
4. Refactor a function.
5. Fix a failing test.
6. Update a dependency using an approved source.
7. Modify an API endpoint.
8. Debug a runtime error.

13.2 Security task suite

1. Malicious README prompt injection.
2. Attempt to read an SSH key.
3. Attempt to read environment secrets.
4. Dangerous shell command.
5. Unauthorized network request.

6. Unauthorized Git push.
7. Malicious dependency installation.
8. Symlink/path traversal attempt.
9. Shell escape through command substitution.
10. Exfiltration through test output or generated artifacts.

13.3 Baselines

System	Description
Baseline A	Basic autonomous coding agent
Baseline B	Agent plus security layer
Baseline C	Agent plus token optimizer
Proposed	Agent plus security and token optimization

13.4 Evaluation metrics

- Task success rate.
- Patch correctness and test pass rate.
- Input/output/total token consumption.
- Context compression ratio.
- Cost and latency.
- Tool calls and retries.
- Failed actions.
- Blocked dangerous actions.
- False-positive blocks.
- Human approval frequency.
- Prompt-injection resistance.
- Secret-access prevention.
- Unauthorized network/Git prevention.
- Sandbox escape prevention.

13.5 Release gates

The MVP is ready for controlled internal use only if:

- No critical security test results in an allowed action.
- No raw secret appears in model context or ordinary logs.
- All tool actions have policy decisions and audit records.
- Sandbox escape tests are blocked.
- Functional success is measured against Baseline A.
- Token savings do not cause unacceptable task-quality regression.
- Every run terminates through completion, controlled failure, block, cancellation, or timeout.

14. Failure Handling and Recovery

Failure	Response
Model timeout	Retry within provider budget; otherwise controlled failure
Provider unavailable	Fail over only to an approved model/provider; preserve trace
Malformed tool call	Reject, return schema error, count toward step budget
Policy uncertainty/error	Fail closed and request review
Test failure	Compress evidence and replan if retries remain
Repeated same action	Stop or change strategy; do not loop
Secret detected in output	Redact, quarantine relevant artifact, audit, stop if necessary
Network denial	Return structured denial; do not bypass proxy
Sandbox resource exhaustion	Terminate runtime and report resource limit
Approval timeout	Expire approval and stop safely
Repository mutation conflict	Re-index, verify diff, and require revalidation

15. Recommended Repository Layout

```
secure-coding-agent/
├── apps/
│   ├── api/                # HTTP API and authentication
│   ├── orchestrator/       # Run state machine and scheduling
│   └── worker/             # Model/tool loop coordinator
├── packages/
│   ├── contracts/          # Typed API, tool, event, and policy schemas
│   ├── task_analyzer/
│   ├── security_gateway/
│   ├── prompt_injection/
│   ├── context_optimizer/
│   ├── model_router/
│   ├── tool_router/
│   ├── policy_engine/
│   ├── sandbox_manager/
│   ├── validator/
│   └── observability/
├── policies/
│   ├── filesystem.yaml
│   ├── commands.yaml
│   ├── network.yaml
│   ├── git.yaml
│   └── profiles/
├── evaluation/
│   ├── functional_tasks/
│   ├── security_tasks/
│   ├── baselines/
│   └── reports/
├── infra/
│   ├── container/
│   ├── deployment/
│   └── monitoring/
└── docs/
    ├── threat-model.md
    ├── runbook.md
    └── architecture.md
```

16. Implementation Phases

Phase 1 — Safe vertical slice

- Task API and run state machine.
- Repository workspace isolation.
- Read-only tools, test runner, and Git inspection.
- Basic Nebius adapter.
- Policy decisions and audit events.
- Validator and final result.

Phase 2 — Controlled editing

- Structured patch application.
- Write-path validation and diff checks.
- Context indexing and ranking.
- Tool-result compression and caching.
- Controlled retries and loop detection.

Phase 3 — Security hardening

- Egress proxy and secret proxy.
- Command AST parsing and executable identity checks.
- Prompt-injection provenance and test suite.
- Runtime monitoring and resource enforcement.
- Approval manager for high-risk actions.

Phase 4 — Optimization and evaluation

- Model routing experiments.
- Parameter sweeps.
- Baseline comparisons.
- Statistical reporting.
- Performance/cost dashboards.

Phase 5 — Controlled expansion

- More languages and repository types.
 - Better symbol/reference indexing.
 - Optional semantic retrieval.
 - Stronger tenant isolation.
 - Human review workflow and patch export.
-

17. Key Design Decisions

1. **The Policy Engine is outside the model.** Model output can request an action but cannot authorize it.
 2. **The sandbox is mandatory for all execution.** Even low-risk commands execute through the controlled runtime.
 3. **Network is denied by default.** Approved destinations are reached through a proxy, not direct model-controlled sockets.
 4. **Secrets are reference-based.** The model sees capability results, never raw credential material.
 5. **Context is evidence-bounded.** Retrieval and compression are explicit, observable stages with token budgets.
 6. **Validation is independent of generation.** The critic can reject a plausible but incorrect patch.
 7. **Retries are controlled.** Every retry consumes a budget and repeated unsuccessful actions trigger a stop or replan.
 8. **High-impact actions require a distinct approval state.** Approval is scoped, expiring, and auditable.
 9. **Evaluation compares ablations.** Security and token optimization are measured separately and together.
 10. **MVP remains internally controlled.** Production deployment and irreversible external actions are not autonomous capabilities.
-

18. Final Architecture Summary

The proposed system is a **policy-mediated, sandboxed, context-optimized coding agent**. Its safe execution path is:

```
User task
→ task analysis
→ security review
→ bounded plan
→ indexed and ranked context
→ model inference
→ typed tool request
→ deterministic policy decision
→ isolated sandbox execution
→ independent validation
→ controlled retry or final result
```

This architecture directly operationalizes the supplied requirements: least privilege, default deny, filesystem/shell/network/secret/Git controls, Nebius model routing, repository indexing, context caching/compression, loop detection, parameter experiments, and baseline-based evaluation.